Computational Physics Laboratory report

Stefano Ge

Winter semester 2025

Contents

0 Introduction **2**

1 Error analysis **2**
1.0.1 Approximating exponentials 2
1.2 Floating-point arithmetic and roundoff errors 3
1.2.1 Computing the Basel problem 3
1.3 Error propagation and condition number 4
1.3.1 Computing statistical momenta 4
1.3.2 Condition number: study of a simple algorithm 4

2 Linear systems **4**
2.1 Forward- and back-substitution 4
2.2 LUP Decomposition 5
2.3 Conditioning of linear systems 6
2.4 Overdetermined linear systems 7
2.4.1 Least squares through normal equations 7
2.4.2 Fitting of Kepler's Law 7
2.4.3 Fitting of exp(sine) function 8
2.5 QR decomposition 10
2.5.1 Least squares through QR 10
2.5.2 QR decomposition for eigenvalues and eigenvectors 11

3 Interpolation **11**

4 Roots of nonlinear equations **11**

5 Numerical integration **11**
5.1 Newton-Cotes formula 11
5.1.1 Trapezoidal rule 11
5.1.2 Simpson's rule 11
5.2 Free-nodes integration 11
5.2.1 Nodes and weights of Gauss-Legendre rule 11
5.2.2 Integrals with Gauss-Legendre rule 11
5.3 Advanced topics in integration 11

6 Ordinary differential equations **11**

0 Introduction

1 Error analysis

1.0.1 Approximating exponentials

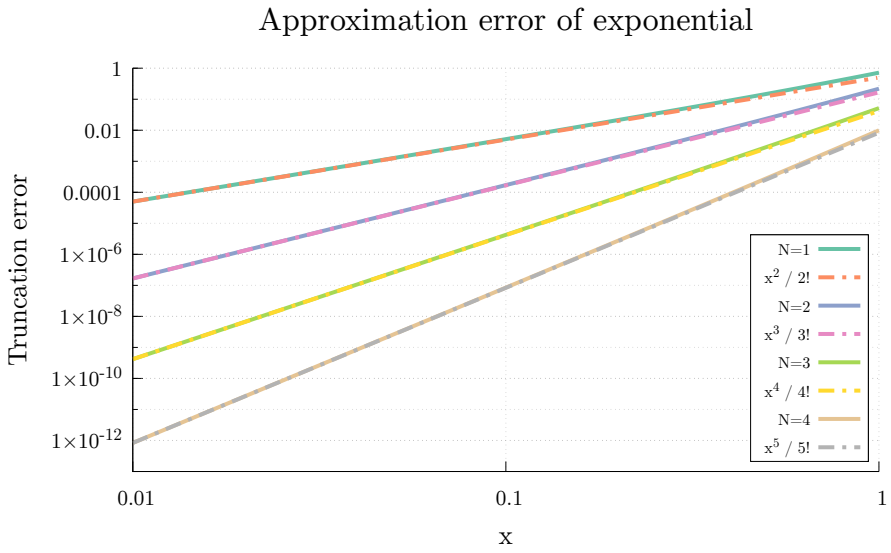


Figure 1: Truncation errors when approximating e^x through the truncated power series Σ_0^N. N is the degree of the polinomyal used for the approximation. The plot of $x^{N+1}/(N+1)$ has been added for reference.

Results

Remarks

- Horner's method was used in polynomial evaluation, because it is faster and reduces roundoff errors. A naive implementation of a polynomial computation requires more operations ($O(N^2)$) and, for small values of x, adds together numbers of order 1 and x^N, while Horner's method is $O(N)$ and keeps the additions only between terms of order 1 and x.

- By Taylor's theorem, if $\hat{f}(x)$ is the power series of e^x truncated after the N-th term, the error is of order $O(x^{N+1})$ as x approaches 0. Computing an additional term of the series shows that:

$$|e^x - \hat{f}(x)| = \frac{x^{N+1}}{(N+1)!} + o(x^{N+1})$$

  It is then not surprising that the approximation error in Figure 1 is indistinguishable from $x^{N+1}/(N+1)$ for small values of x. For larger values, the contribution of higher order terms in the series becomes ever more important, and hence the divergence from $x^{N+1}/(N+1)$.

1.2 Floating-point arithmetic and roundoff errors

1.2.1 Computing the Basel problem

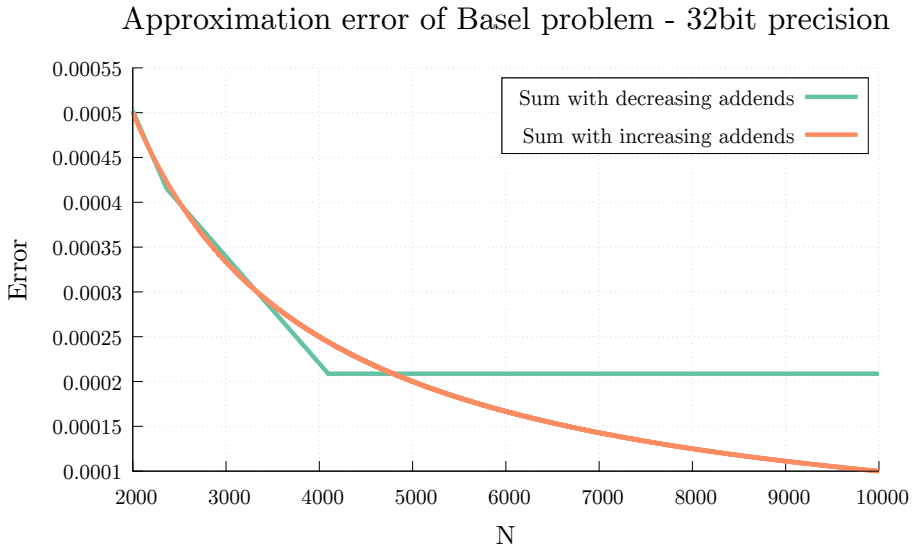


Figure 2: Approximation error when computing $\pi^2/6$ with the Basel problem truncated series $\Sigma_0^N \frac{1}{n^2}$, with single precision floating variables. Here N is the number of terms used when calculating the series.

Results

Remarks

- The algorithm has been implemented with two mathematically equivalent forms, which, however, resulted in remarkably different numerical results. In the first the terms of the series are summed in increasing order of the summation index, i.e. decreasing of the value of the addends. Since small addends are summed to the accumulating partial sum, it gets to a point where the next addend is smaller than $\epsilon_{mach} \cdot sum \approx \epsilon_{mach}$, so the addition of it and subsequent values does not change the value of the partial sum.
  This can be seen in the error plots in Figure 2 and 3, since the error stops decreasing and remains constant after $1/\mathrm{N}^2 = \epsilon_{\mathrm{mach}}$, which is 2^{-24} for single precision ($\mathrm{N} = 4096$) and 2^{-53} for double precision ($\mathrm{N} \approx 9.5 \times 10^7$).

  In the second implementation the terms of the series are summed in decreasing order of the summation index, i.e. increasing the value of the addends. Therefore, the aforementioned effect is not seen, and the numerical results correspond to the mathematical expectation, in the range considered.

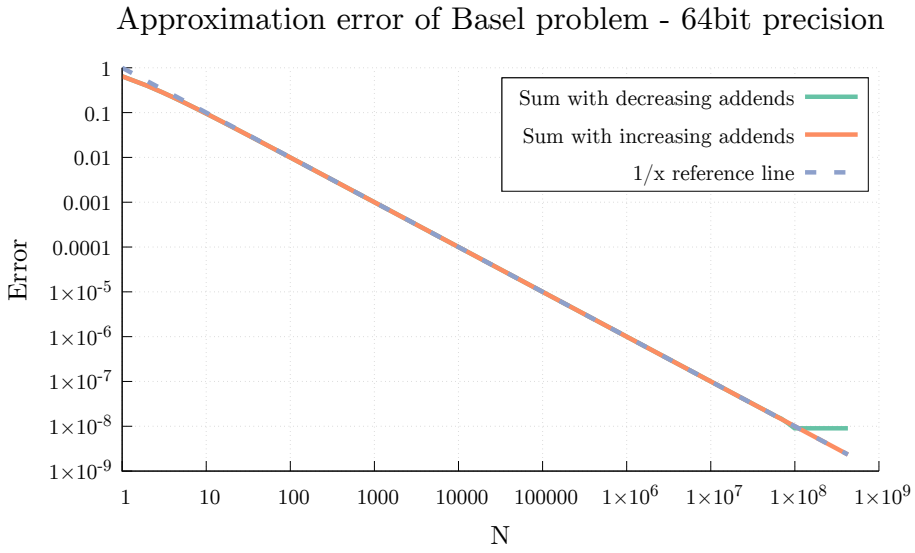


Figure 3: Same as above, but now using double precision floating variables. the plot of $1/x$ has been added for reference.

- The truncation error of this algorithm is asymptotically equivalent to k/N, with k real constant. This can be from the integral associated to $\Sigma_{n=0}^{\infty}$:

$$\int_N^{\infty} \frac{1}{n^2} = \frac{1}{N}$$

therefore,

$$\Sigma_{n=N}^{\infty} \sim_{N\to\infty} \frac{1}{N}$$

which is confirmed from the agreement between the computed values and the $1/x$ reference line in Figure 3.

1.3 Error propagation and condition number

1.3.1 Computing statistical momenta

1.3.2 Condition number: study of a simple algorithm

2 Linear systems

2.1 Forward- and back-substitution

Results

$$\begin{bmatrix} -2 & 0 & 0 \\ 1 & -1 & 0 \\ 3 & 2 & 1 \end{bmatrix} \mathbf{X} = \begin{bmatrix} -4 \\ 2 \\ 1 \end{bmatrix} \qquad \mathbf{X} = \begin{bmatrix} 2 \\ 0 \\ -5 \end{bmatrix}$$

$$\begin{bmatrix} 4 & 0 & 0 & 0 \\ 1 & -2 & 0 & 0 \\ -1 & 4 & 4 & 0 \\ 2 & -5 & 5 & 1 \end{bmatrix} \mathbf{X} = \begin{bmatrix} -4 \\ 1 \\ -3 \\ 5 \end{bmatrix} \qquad \mathbf{X} = \begin{bmatrix} -1 \\ -1 \\ 0 \\ 2 \end{bmatrix}$$

$$\begin{bmatrix} 3 & 1 & 0 \\ 0 & -1 & -2 \\ 0 & 0 & 3 \end{bmatrix} \mathbf{X} = \begin{bmatrix} 1 \\ 1 \\ 6 \end{bmatrix} \qquad \mathbf{X} = \begin{bmatrix} 2 \\ -5 \\ 2 \end{bmatrix}$$

$$\begin{bmatrix} 3 & 1 & 0 & 6 \\ 0 & -1 & -2 & 7 \\ 0 & 0 & 3 & 4 \\ 0 & 0 & 0 & 5 \end{bmatrix} \mathbf{X} = \begin{bmatrix} 4 \\ 1 \\ 1 \\ 5 \end{bmatrix} \qquad \mathbf{X} = \begin{bmatrix} -3.\overline{3} \\ 8 \\ -1 \\ 1 \end{bmatrix}$$

The results obtained through my routine were verified using LAPACK.

2.2 LUP Decomposition

Results

$$\begin{bmatrix} 2 & 3 & 4 \\ 4 & 5 & 10 \\ 4 & 8 & 2 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 0.5 & 0.\overline{3} & 1 \end{bmatrix} \begin{bmatrix} 4 & 8 & 2 \\ 0 & -3 & 8 \\ 0 & 0 & 0.\overline{3} \end{bmatrix}$$

Determinant is 4.

$$\begin{bmatrix} 6 & -2 & -4 & 4 \\ 3 & -3 & -6 & 1 \\ -12 & 8 & 21 & -8 \\ -6 & 0 & -10 & 7 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0.5 & 1 & 0 & 0 \\ -0.25 & 0.25 & 1 & 0 \\ -0.5 & -0.5 & -0.857143 & 1 \end{bmatrix} \begin{bmatrix} -12 & 8 & 21 & -8 \\ 0 & -4 & -20.5 & 11 \\ 0 & 0 & 4.375 & -3.75 \\ 0 & 0 & 0 & 2.28571 \end{bmatrix}$$

Determinant is -480.

$$\begin{bmatrix} 1 & 4 & 5 & -5 \\ -1 & 0 & -1 & -5 \\ 1 & 3 & -1 & 2 \\ 1 & -1 & 5 & -1 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 0.8 & 1 & 0 \\ -1 & -0.2 & -0.\overline{6} & 1 \end{bmatrix} \begin{bmatrix} 1 & -1 & 5 & -1 \\ 0 & 5 & 0 & -4 \\ 0 & 0 & -6 & 6.2 \\ 0 & 0 & 0 & -2.\overline{6} \end{bmatrix}$$

Determinant is 80.

Matrix $\mathbf{A} = \mathbf{T}(3, -1)\mathbf{R}(\pi/5)\mathbf{T}(-3, 1)$:

$$A = \begin{bmatrix} 2.3 & 1.6 & -2.3 \\ -1.1 & 0.48 & 2.7 \\ 0.5 & 0.\overline{3} & -0.\overline{6} \end{bmatrix}$$

$$\mathbf{b} = \mathbf{Az} = \mathbf{A} \begin{bmatrix} 2 \\ 2 \\ 1 \end{bmatrix} = \begin{bmatrix} 5.45 \\ 1.51 \\ 1.5 \end{bmatrix}$$

Factorization of $\mathbf{A}$ is:

$$\mathbf{A} = \begin{bmatrix} 1 & 0 & 0 \\ -0.471 & 1 & 0 \\ 0.217 & -0.00857 & 1 \end{bmatrix} \begin{bmatrix} 2.31 & 1.59 & -2.34 \\ 0 & 1.22 & 1.64 \\ 0 & 0 & 0.354 \end{bmatrix}$$

2.3 Conditioning of linear systems

Results In the case study proposed, first we construct numerically:

$$\mathbf{b} = \mathbf{A}\mathbf{x} = \begin{bmatrix} -\epsilon & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

Where ϵ is a real parameter. We then apply the LU decomposition (without pivoting) to solve for x given $\mathbf{A}$ and the $\mathbf{b}$ just computed.

For $\epsilon = 10^{-12}$ we find:

$$\mathbf{A} = \mathbf{L}\mathbf{U} = \begin{bmatrix} 1 & 0 \\ -1\text{e}{-12} & 1 \end{bmatrix} \begin{bmatrix} -\text{e}{-12} & 1 \\ 0 & 1\text{e}12 \end{bmatrix}$$

$$\mathbf{x}_{\text{computed}} - \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} -2.2\text{e}{-5} \\ 0 \end{bmatrix}$$

As a test, we find correctly:

$$\mathbf{A} - \mathbf{L}\mathbf{U} = \mathbf{0}$$

However, if we set $\epsilon = 10^{-12}$, we find:

$$\mathbf{A} = \mathbf{L}\mathbf{U} = \begin{bmatrix} 1 & 0 \\ -1\text{e}{-20} & 1 \end{bmatrix} \begin{bmatrix} -\text{e}{-20} & 1 \\ 0 & 1\text{e}20 \end{bmatrix}$$

$$\mathbf{x}_{\text{computed}} - \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} -1 \\ 0 \end{bmatrix}$$

$$\mathbf{A} - \mathbf{L}\mathbf{U} = \begin{bmatrix} 0 & 0 \\ 0 & -1 \end{bmatrix}$$

Remarks

- The exact LU-factorization of $\mathbf{A}$ is:

  $$\mathbf{A} = \begin{bmatrix} 1 & 0 \\ -\epsilon^{-1} & 1 \end{bmatrix} \begin{bmatrix} -\epsilon & 1 \\ 0 & \epsilon^{-1} - 1 \end{bmatrix}$$

  If $\epsilon = 10^{-20} \ll \epsilon_{\text{mach}}$, in the bottom-right entry of $\mathbf{U}$ only ϵ^{-1} gets stored, since the relative variation of adding -1 to ϵ^{-1} is less than ϵ_{mach}. But when $\mathbf{L}$ and $\mathbf{U}$ get multiplied together, in the addition resulting into the bottom-right entry of $\mathbf{A}$ two ϵ^{-1} terms cancels out, and the exact result is the -1 that is lost in numerical LU-decomposition.

- Exact $\mathbf{b}$ is:

  $$\begin{bmatrix} 1 - \epsilon \\ 0 \end{bmatrix}$$

  but ϵ does not get registered if it is lower than ϵ_{mach}. Because the condition number of the matrices $\mathbf{L}$ and $\mathbf{U}$ is very high (ϵ^{-2} in ∞-norm), the tiny variation of $\mathbf{b}_{\text{calc}}$ over $\mathbf{b}_{\text{exact}}$ is allowed to propagate tremendously and results in the wrong result seen before.

  Doing the calculations shows that the error on $\mathbf{U}$ as no impact on $\mathbf{b}_{\text{calc}}$, and that the final error on $\mathbf{b}_{\text{calc}}$ is due entirely on the ill-conditioning of the linear systems $\mathbf{L}\mathbf{z} = \mathbf{b}$ and $\mathbf{U}\mathbf{x} = \mathbf{z}$.

- By contrast, the condition number of the starting matrix $\mathbf{A}$ is only ≈ 4. Therefore, in principle it should be possible to solve the system for $\mathbf{x}$ even if $\mathbf{b}$ is slightly nudged: the problem is the stability of the algorithm adopted.

  LU-decomposition without row-pivoting is much less numerically stable than LUP-decomposition with row-pivoting where the maximum value entry in a column is selected as the pivot.

  Indeed, if we perform LU-decomposition on the following matrix, obtained by permuting the two rows of matrix $\mathbf{A}$, as prescribed by row-pivoting, we find the correct results:

$$\mathbf{PA} = \begin{bmatrix} 1 & -1 \\ -\epsilon & 1 \end{bmatrix} = \mathbf{LU} = \begin{bmatrix} 1 & 0 \\ -1\text{e}{-20} & 1 \end{bmatrix} \begin{bmatrix} ta1 & -1 \\ 0 & 1 \end{bmatrix}$$

$$\mathbf{x}_{\text{computed}} - \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

$$\mathbf{A} - \mathbf{LU} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$$

2.4 Overdetermined linear systems

2.4.1 Least squares through normal equations

Results After implementing a routine for finding a the vector $\mathbf{X}$ that best fits an overdetermined linear system, we tested it on the following linear system:

$$\begin{bmatrix} 2 & -1 \\ 0 & 1 \\ -2 & 2 \end{bmatrix} \mathbf{X} = \begin{bmatrix} 1 \\ -5 \\ 6 \end{bmatrix}$$

and found:

$$\mathbf{X} = \begin{bmatrix} -2 \\ 1 \end{bmatrix}$$

2.4.2 Fitting of Kepler's Law

We want to extrapolate Kepler's third law from experimental data through the algorithm just introduced. First we linearize the following relationship by taking the logarithm at both sides:

$$\mathrm{T} = \mathrm{CD}^{\alpha} \implies \log(\mathrm{T}) = \log(\mathrm{C}) + \alpha \log(\mathrm{D})$$

where T is the orbital periods of the planets, D is their average distance from the Sun and alpha and C are two costants to be determined through the method of least squares. This is equivalent to finding x_1 and x_2 that minimize:

$$\log(\mathbf{T}) = x_1\mathbf{1} + x_2 \log(\mathbf{D}) = \begin{bmatrix} \mathbf{1} & \log(\mathbf{D}) \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

where quantities in bold are vectors, leading to an overdetermined linear system.

Results We therefore find that the values that fit the aforementioned system are:

$$\alpha = 1.49865 \qquad C = 0.201522$$

where alpha is consistent with Kepler's law, which predicts $\alpha = 1.5$.

The graph containing the fitted curve and experimental data can be found in Figure 4

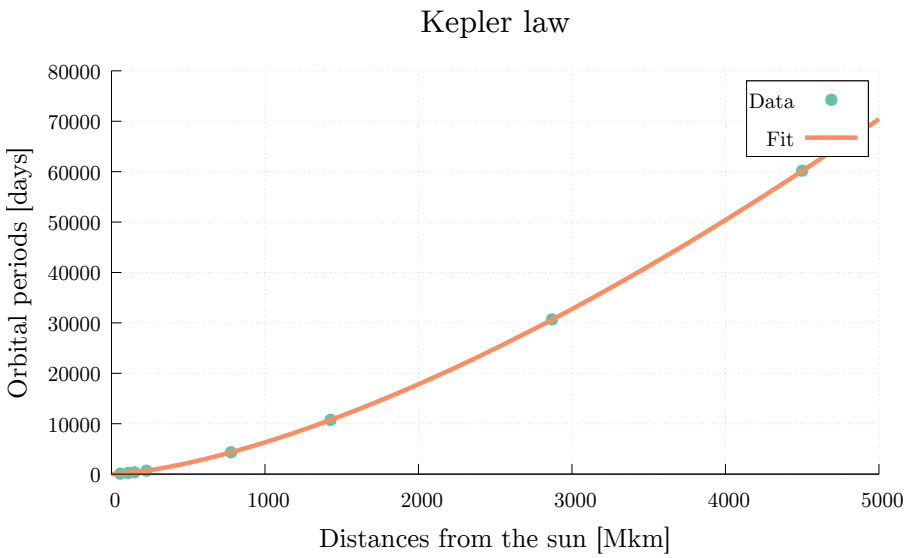


Figure 4: Orbital peirods vs distances from the Sun of the planets.

2.4.3 Fitting of exp(sine) function

Results We want to approximate the function $\exp(\sin(t-1))$, first through a Taylor-like polynomial:

$$y(t) = c_1 + c_2 t + ... + c_7 t^6$$

and then through a periodic Fourier-like approximation:

$$y(t) = d_1 + d_2 \cos(t) + d_3 \sin(t) + d_4 \cos(2t) + d_5 \sin(5t)$$

Results are shown in Figure 5 and in Figure 6.

Remarks

- Not surpringly, the function is better approximated by the first terms of the Fourier basis than by a sixth degree polynomial. The Fourier basis is an orthonormal complete set of $L^2([0, 2\pi])$, therefore it is guaranteed to be the best approximation in L^2 norm, among expansions on sets spanned by it. Moreover, sines and cosines are themselves periodic and are thus well suited for approximating a periodic function like $\exp^{\sin(t-1)}$, producing a nice approximation over the whole domain $\mathbb{R}$. On the other hand, monic

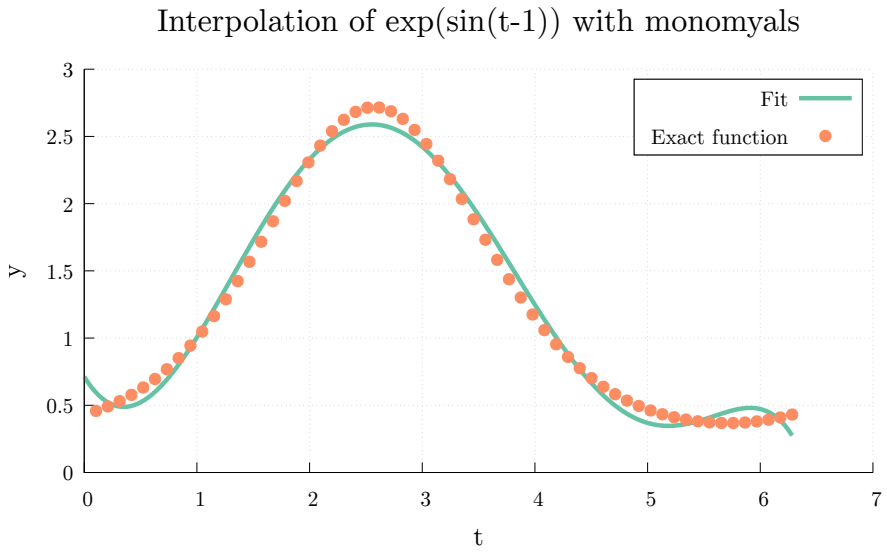


Figure 5: Least square fit of $\exp(\sin(t-1)$ with 6th degree polynomial

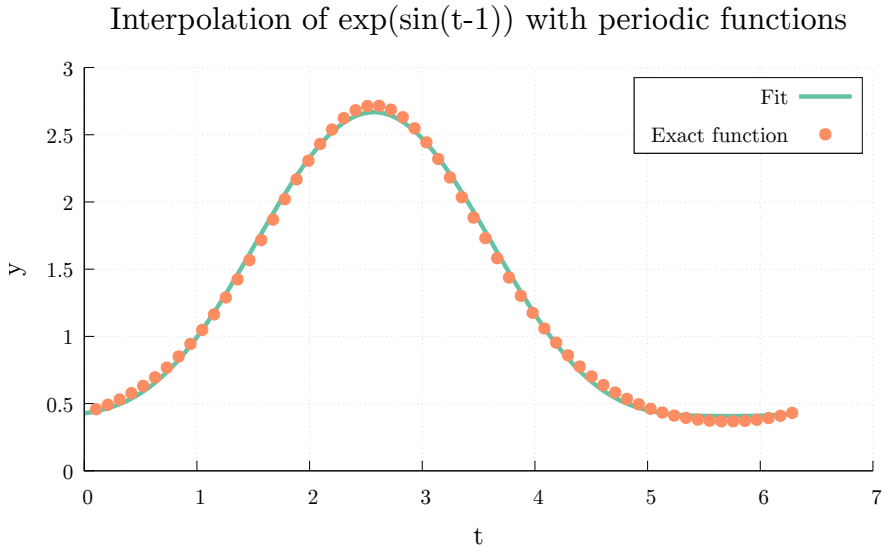


Figure 6: Least square fit of $\exp(\sin(t-1)$ with Fourier periodic functions

monomyals, though complete, are neither an orthonormal set of $L^2([0, 2\pi])$ nor are well suited for approximating periodic functions, as they do blow up at infinity and do not approximate well outside of the points employed in the fit.

2.5 QR decomposition

In this section we implement algorithms to find the QR decomposition of any matrices. First the modified Gram-Schmidt procedure (MGS).

Results We test the modified Gram-Schmidt procedure on the following matrices:

$$\begin{bmatrix} 2 & -1 \\ 0 & 1 \\ -2 & 2 \end{bmatrix} = \begin{bmatrix} 0.707 & 0.408 \\ 0.000 & 0.816 \\ -0.707 & 0.408 \end{bmatrix} \begin{bmatrix} 2.828 & -2.121 \\ 0 & 1.225 \end{bmatrix}$$

$$\begin{bmatrix} 2 & 3 & 4 \\ 4 & 5 & 10 \\ 4 & 8 & 2 \end{bmatrix} = \begin{bmatrix} 0.333 & -0.104 & 0.937 \\ 0.667 & -0.677 & -0.312 \\ 0.667 & 0.729 & -0.156 \end{bmatrix} \begin{bmatrix} 6.000 & 9.667 & 9.333 \\ 0 & 2.134 & -5.726 \\ 0 & 0 & 0.312 \end{bmatrix}$$

$$\begin{bmatrix} 2 & -1 & 4 \\ 0 & 3 & 5 \\ 7 & 2 & -6 \\ 1 & -4 & 0 \end{bmatrix} = \begin{bmatrix} 0.272 & -0.241 & 0.733 \\ 0.000 & 0.559 & 0.617 \\ 0.953 & 0.179 & -0.233 \\ 0.136 & -0.773 & 0.163 \end{bmatrix} \begin{bmatrix} 7.348 & 1.089 & -4.627 \\ 0 & 5.368 & 0.752 \\ 0 & 0 & 7.418 \end{bmatrix}$$

Computing the product $\mathbf{A} = \mathbf{QR}$ gives us the matrices we have decomposed, confirming the correctness of the decomposition. Also we computed $\mathbf{Q}^\intercal\mathbf{Q}$ and we confirmed that the computed $\mathbf{Q}$'s are indeed ONC matrices.

2.5.1 Least squares through QR

We implemented QR algorithm to solve least-squares problem, and applied it to the problem of fitting the function $\exp^{\sin(4t)}$ with a 14th degree polynomial.

Results

2.5.2 QR decomposition for eigenvalues and eigenvectors

3 Interpolation

4 Roots of nonlinear equations

5 Numerical integration

5.1 Newton-Cotes formula

5.1.1 Trapezoidal rule

5.1.2 Simpson's rule

5.2 Free-nodes integration

5.2.1 Nodes and weights of Gauss-Legendre rule

Remarks

5.2.2 Integrals with Gauss-Legendre rule

5.3 Advanced topics in integration

6 Ordinary differential equations